Crittografia con OpenSSL – Operazioni di base

Laboratorio del corso "Sicurezza dei sistemi informativi" (01UDUOV)
Politecnico di Torino – AA 2023/24
Prof. Cataldo Basile

preparata da:
Gabriele Gatti (gabriele.gatti@polito.it)

v. 1.1 (31/10/2023)

Indice

1 Algoritmi di cifratura simmetrica **7**
1.1 Cifratura simmetrica . . . 7
1.2 Attacco a forza bruta . . . 8
1.3 Prestazioni . . . 9

2 Algoritmi di cifratura asimmetrica **10**
2.1 Generazione di chiavi RSA . . . 10
2.2 Operazioni di cifratura e decifratura asimmetrica con l'algoritmo RSA . . . 10
2.3 Operazioni di firma e verifica con l'algoritmo RSA . . . 12
2.4 Generazione di chiavi EC . . . 12
2.5 Operazioni di firma e verifica EC . . . 12
2.6 Prestazioni . . . 12

3 Algoritmi di digest **13**
3.1 Calcolare e verificare il digest di un messaggio . . . 13
3.2 Collisioni . . . 14
3.3 Prestazioni . . . 14

4 Approfondimenti ed ulteriori esercizi (opzionali) **15**
4.1 Modifica dei messaggi cifrati con crittografia simmetrica . . . 15
4.2 Uso degli algoritmi di cifratura a blocchi . . . 16
4.3 Integrità di file e cartelle . . . 16

Scopo dell'esercitazione

Scopo dell'esercitazione è sperimentare le procedure e le problematiche associate all'uso delle diverse primitive crittografiche. L'esercitazione è basata sull'uso estensivo di OpenSSL (http://www.openssl.org/), una libreria crittografica rilasciata come software *open source* con licenza Apache e disponibile su un ampio insieme di piattaforme, Linux e Windows inclusi.

L'esercitazione è basata sull'uso di OpenSSL da linea di comando che permette di invocare varie funzioni crittografiche attraverso la shell digitando:

```
openssl command [ command_opts ] [ command_args ]
```

Per il corretto svolgimento di questa esercitazione sarà necessario usare i comandi:

```
enc genrsa rsa ecparam ec pkeyutl dgst rand speed
```

che saranno presentati di seguito. Per la lista completa delle opzioni ed una descrizione dettagliata dei comandi consigliamo di visionare le relative pagine `man`.

I file ed i programmi necessari allo svolgimento di alcuni singoli esercizi sono disponibili nell'archivio

02lab-materiale.zip

Per decomprimerli potete usare il seguente comando:

```
7z x "02lab - materiale.zip"
```

openssl enc

Il comando `enc` permette di cifrare e decifrare blocchi di dati usando un gran numero di algoritmi crittografici. Potete ottenere informazioni su questo comando digitando:

```
man enc
```

Ad esempio, per visualizzare la lista degli algoritmi supportati dal comando `enc`, si può eseguire il comando:

```
openssl enc -ciphers
```

oppure

```
openssl list -cipher-algorithms
```

Qui di seguito riportiamo la sintassi generale ed i parametri più importanti (si ricorda che i comandi in parentesi quadre sono opzionali):

```
openssl enc [-algoritmo_di_encryption] [-e] [-d] [-K chiave] [-iv vettore]
  [-in file_in] [-out file_out] [-nopad] [-p]
```

in cui:

- *-algoritmo_di_encryption*, serve ad indicare l'algoritmo da usare per svolgere le operazioni di crifratura simmetrica (es. `-aes128`, `-aes-256-cbc`, `-rc4`, la lista completa si ha con `openssl enc -ciphers`);
- `-e` richiede di svolgere la cifratura (encryption) e naturalmente esclude l'opzione `-d`;
- `-d` richiede di svolgere la decifratura e naturalmente esclude l'opzione `-e`;
- `-K` *chiave*, richiede l'uso della chiave *chiave* per le operazioni di crittografia simmetrica;

ATTENZIONE

In `openssl`, le opzioni sono case-sensitive e per questa esercitazione è importante distinguere la -K "maiuscola" dalla -k "minuscola" che servirà in un solo esempio.

- `-iv` *vettore*, per usare il vettore di inizializzazione *vettore*;
- `-in` *file_in* indica che il file *file_in* contiene i dati da cifrare o decifrare;
- `-out` *file_out* per salvare in *file_out* l'output di `enc`;
- `-nopad`, non applica il padding.

ATTENZIONE

Se si usa `-nopad` con algoritmi a blocchi, la lunghezza del dato da cifrare deve essere necessariamente un multiplo della lunghezza del blocco o l'operazione fallirà. Non ha senso e viene ignorato invece con algoritmi di tipo stream.

- `-p`, per stampare su standard output la chiave ed il vettore di inizializzazione usati (in realtà stampa sempre anche il cosiddetto *salt* generato in maniera pseudo-casuale da OpenSSL, anche se non necessariamente verrà usato).

openssl dgst

Il comando `dgst` permette di calcolare il digest di un blocco di dati usando uno dei diversi algoritmi di digest supportati dalla libreria OpenSSL. Per una descrizione approfondita del comando `dgst` potete richiamare la sua pagina man:

```
man dgst
```

Per visualizzare la lista degli algoritmi supportati dal comando `dgst` potete eseguire il comando:

```
openssl list -digest-commands
```

Qui di seguito la sintassi generale ed i parametri più importanti:

```
openssl dgst [-algoritmo_di_digest] [-out file_out] file_in(s)
```

in cui:

- *-algoritmo_di_digest*, serve ad indicare l'algoritmo da usare per svolgere le operazioni di digest (es. `-md5`, `-sha1`, la lista completa si ha con `openssl list -digest-commands`);
- `-out` *file_out*, per salvare in *file_out* il digest prodotto;
- *file_in(s)*, per indicare i file di cui calcolare il digest.

ATTENZIONE

Il comando `dgst` non usa l'opzione `-in`, ma i file di cui calcolare il digest sono passati come ultimi argomenti sulla riga di comando

openssl genrsa

Per generare delle chiavi RSA, OpenSSL mette a disposizione il comando `genrsa`:

```
openssl genrsa [-out file_out] [numbits]
```

in cui:

- `-out` *file_out* indica che le chiavi generate (pubblica e privata) saranno salvate nel file *file_out*;
- *numbits* specifica la lunghezza (in bit) del modulo di RSA.

> NOTA
>
> Quando il `man` di OpenSSL parla di chiave privata, intende solitamente l'insieme di chiave privata e chiave pubblica, dato che sono entrambe contenute nella stessa struttura dati. Volendo, le componenti private di una chiave RSA potrebbero essere conservate separatamente da quelle pubbliche, ma ciò non ha molto senso, è meno efficiente dal punto di vista computazionale e viene fatto raramente (cfr. lo standard PKCS#1).

openssl rsa

Per gestire e manipolare le chiavi RSA, si può usare il modulo `rsa`:

```
openssl rsa [-in file_in] [-out file_out] [-text] [-pubin] [-pubout] [-noout]
```

in cui:

- `-in` *file_in*, specifica il file di input (contenente una chiave pubblica o privata);
- `-out` *file_out*, salva in *file_out* le chiavi (pubblica o privata) dopo aver svolto l'operazione richiesta;
- `-text`, visualizza le chiavi in formato testuale, utile ai fini di visualizzazione. Le chiavi saranno visualizzate anche in formato Base64 a meno che non venga anche usato `-noout`;
- `-pubin`, flag che indica che la chiave passata in input (mediante il comando `-in`) è una chiave pubblica (se non specificato, `rsa` assume che sia una chiave privata);
- `-pubout`, serve a produrre in output la chiave pubblica, più precisamente, produce in output solo le componenti pubbliche (se non specificato, `rsa` produrrà la chiave privata, cioè tutto);
- `-noout`, blocca la produzione in output delle chiavi in formato Base64.

openssl ecparam

Per gestire e manipolare i parametri degli algoritmi su curve ellittiche si può usare il modulo `ecparam`:

```
openssl ecparam [-list_curves] [-name curva] [-genkey] [-out -file_out]
```

in cui:

- `-list_curves`, elenca le curve disponibili;
- `-name` *curva* specifica una curva per nome;
- `-genkey` fa generare le chiavi pubbliche e private;

- `-out` *file_out*, salva in *file_out* le chiavi (pubblica o privata).

openssl ec

Per gestire e manipolare le chiavi degli algoritmi su curve ellittiche si può usare il modulo `ec`:

```
openssl ec [-in file_in] [-out file_out] [-pubout] [-pubin] [-text]
```

in cui:

- `-in` *file_input*, specifica il file di input (che contiene le chiavi da leggere);
- `-out` *file_out*, salva in *file_out* la chiave estratta dal file di input;
- `-pubout`, indica di produrre in uscita la chiave pubblica (se non è specificato il comando produce la chiave privata);
- `-pubin`, indica di leggere la chiave pubblica (se non è specificato il comando legge la chiave privata);
- `-text`, visualizza le chiavi in formato testuale, utile ai fini di visualizzazione.

openssl pkeyutl

Il comando `pkeyutl` serve ad effettuare operazioni di cifratura, decifratura, firma, verifica e scambio di chiavi, mediante diversi algoritmi asimmetrici. Attualmente gli algoritmi supportati sono:

- RSA, per cifrare, decifrare, firmare e verificare;
- DSA, per firmare e verificare;
- Diffie-Hellman (DH), per scambiare una chiave simmetrica;
- Elliptic Curve (EC) algorithms per firmare e verificare usando ECDSA o scambiare una chiave simmetrica con ECDH.

```
openssl pkeyutl [-encrypt] [-decrypt] [-sign] [-verify] [-verifyrecover]
  [-in file_in] [-out file_out]
  [-pubin] [-inkey file_chiave] [-sigfile signature]
```

in cui:

- `-encrypt/-decrypt`, cifra con la chiave pubblica o decifra con la chiave privata il file di input (passato con `-in`);
- `-sign`, genera la firma applicata al file di input (passato con `-in`) usando la chiave passata con l'opzione `-inkey`. È necessaria una chiave privata. Più precisamente, se si usa una chiave privata RSA, il file passato in input viene cifrato con la chiave privata, altrimenti l'operazione fallisce;
- `-verify`, calcola la firma del file di input (passato con `-in`) usando la chiave pubblica passata con l'opzione `-inkey` e la confronta con un'altra firma passata tramite l'opzione `-sigfile`. Se il file contiene una coppia di chiavi pubblica e privata, verrà usata solo la componente pubblica. Restituisce un valore Booleano (*Signature Verified Successfully/Signature Verification Failure*);
- `-verifyrecover`, verifica la firma passata tramite il file di input (passato con `-in`) usando la chiave passata con l'opzione `-inkey` e visualizza i dati decifrati. È necessaria una chiave pubblica

RSA. Questa opzione è disponibile solo se si usa una chiave RSA ed, in pratica, il file di input viene decifrato con la chiave pubblica (non funziona con DSA, DH o EC);

- `-in` *file_in*, specifica il file di input (che contiene il messaggio da cifrare, decifrare, firmare o verificare);
- `-out` *file_out*, salva in *file_out* l'output di `pkeyutl`;
- `-inkey` *file_chiave*, specifica che il file *file_chiave* contiene la chiave pubblica o privata;
- `-pubin`, flag che indica che la chiave passata in input (mediante il comando `-inkey`) è una chiave pubblica. Se non specificato, `pkeyutl` assume che sia una chiave privata;
- `-sigfile` *firma*, specifica che la firma da confrontare con l'opzione `-verify` è memorizzata nel file *firma*.

ATTENZIONE

Al contrario di tutti gli altri comandi openssl, in `pkeyutl` l'ordine in cui vengono passati i parametri è importante. Rispettate l'ordine dei parametri presentato qui sopra oppure l'esecuzione dei comandi potrebbe fallire.

openssl speed

Col comando `speed` si possono verificare le prestazioni di uno degli algoritmi disponibili in OpenSSL:

```
openssl speed [nome_algoritmo1 nome_algoritmo2 ...]
```

Se non specificate nessun nome di algoritmo, saranno verificate le prestazioni di tutti gli algoritmi in OpenSSL (questo processo può essere lungo).

openssl rand

Col comando `rand` si possono generare `numbyte` casuali e salvarli nel file `file_out`:

```
openssl rand -out file_out numbyte
```

Altri comandi

Nel corso dell'esercitazione potreste voler scambiare dati da un computer all'altro col comando `scp`. Abilitate quindi il server `ssh` su uno dei computer (e.g. Alice):

```
systemctl {status | start | restart | stop } ssh
```

Per creare un nuovo utente con username *alice* potete usare il comando:

```
adduser alice
```

A questo punto, un altro utente da remoto può trasferire il file `prova` sul nodo di Alice usando il comando seguente (da notare i ':' alla fine del comando):

```
scp prova alice@ip_alice:
```

Alice potrà quindi usare il file `prova` che si troverà nella sua directory home (`/home/alice`).

1 Algoritmi di cifratura simmetrica

1.1 Cifratura simmetrica

Provate a cifrare un blocco di dati col comando OpenSSL `enc`. Scegliete una chiave di 32 cifre esadecimali (corrispondenti a 128 bit) e scrivetela qui:

→

Create un file di testo chiamato `ptext` contenente il messaggio da cifrare, ad esempio il seguente testo:

```
Questo e' il mio grande segreto
```

Dopo averlo salvato, verificate che sia lungo esattamente 32 byte (`ls -l`). Pur non essendo indispensabile per il prossimo esercizio, lo sarà in uno degli esercizi successivi.

Usate AES (con chiave di 128 bit) per cifrare il file `ptext` con la chiave da voi scelta. Il messaggio cifrato deve essere salvato in un file chiamato `ctext.aes128`.

Scoprite la stringhe di comando OpenSSL da usare per svolgere l'operazione richiesta e scrivetela sotto. Fate in modo che il comando visualizzi la chiave e l'IV (se l'algoritmo usa un IV) effettivamente usati:

→

Scoprite la stringhe di comando OpenSSL da usare per cifrare `ptext` con gli algoritmi DES, RC4 e 3DES con la chiave da voi scelta. Il messaggio cifrato deve essere salvato in un file chiamato `ctext.`*algoritmo*.

→

Quali altre scelte dobbiamo compiere nel caso di DES, 3DES e AES?

→

Confrontate la lunghezza dei file `ctext.*` generati nei vari casi e quella di `ptext`: motivate le differenze osservate.

→

Provate adesso cosa succede usando il seguente comando.

```
openssl enc -e -in ptext -out ctext.aes128.nopad -K chiave -iv IV -aes-128-cbc
-nopad -p
```

Nel caso abbiate usato un messaggio in chiaro differente, provate ad eseguire l'operazione col messaggio sopra suggerito. Cosa succede quando il messaggio da cifrare ha dimensioni diverse?

→

Spiegare la differenza tra cifrare usando il comando precedente e cifrare con l'algoritmo RC4.

→

Scoprite le stringhe di comando OpenSSL da usare per cifrare `ptext` con l'algoritmo ChaCha20 e una chiave da voi scelta. Spiegare la differenza tra cifrare usando il comando precedente e cifrare con l'algoritmo ChaCha20.

→

Ora provate a decifrare i file `ctext`.*algoritmo* e salvate i messaggi decifrati in nuovi file che potete chiamare `dtext`.*algoritmo*. Scoprite la stringa di comando OpenSSL da usare per la decifratura e scrivetela:

→

Verificate che la decifratura sia stata eseguita correttamente.

Cosa succede usando il seguente comando?

```
openssl enc -e -in ptext -out ctext -aes-128-cbc -p
```

Come è stata calcolata la chiave di cifratura? E l'IV?

→

1.2 Attacco a forza bruta

A questo punto non resta altro che provare a rompere un algoritmo crittografico con un attacco a forza bruta.

1. Eseguite lo script `Es1.2_Alice_Secret_Message.py` contenuto in `02lab - materiale.zip` tramite il comando:

   ```
   python3 Es1.2_Alice_Secret_Message.py
   ```

   esso genererà il file `alice_ctext` nella cartella corrente. Il file contiene il messaggio di Alice criptato con una chiave composta da un singolo carattere esadecimale (4 bit) e il cifrario RC4.

2. Cercate ora di scoprire la chiave usata da Alice per cifrare il messaggio. Siete riusciti a scoprirla?

   NOTA

   Per svolgere il punto 2 non è necessario preparare uno script, dato il numero limitato di tentativi che Bob dovrà fare. Essendo un'esercitazione di laboratorio, a meno che non siate veramente veloci ad automatizzare con qualche linguaggio di scripting, potete scegliere diversi valori della chiave cambiando direttamente il testo sulla riga di comando.

→

Ora provate a scoprire la chiave usata per cifrare il file di test `bruteforce_enc`, che potete trovare nell'archivio `02lab - materiale.zip`. Questo file è stato cifrato secondo la stessa procedura precedentemente seguita da Alice, cioè, con una chiave di un carattere esadecimale ed il comando OpenSSL:

```
openssl enc -e -in ptext -out ctext_alice -K carattere_esadecimale -rc4
```

Incontrate delle difficoltà?

→

1.3 Prestazioni

Soffermiamoci ora a valutare il costo degli algoritmi crittografici in termini sia di tempo impiegato per le operazioni crittografiche sia di quantità di dati addizionali introdotti.

Create alcuni file di dimensioni differenti; ad esempio 100 B, 10 kB, 1 MB e 100 MB. Per creare file di una determinata lunghezza potete usare il comando:

```
openssl rand -out r.txt dimensione_in_byte
```

Ora compilate la Tabella 1 con i tempi necessari per cifrare i file con diversi algoritmi e diverse lunghezze di chiave. Per valutare il tempo impiegato dalle operazioni di cifratura usate il comando `time`:

```
time stringa_di_comando_openssl
```

Qual è la quantità significativa tra quelle riportate dal comando `time`?

→

Il tempo impiegato dall'operazione di decifratura per gli algoritmi in tabella sarà sensibilmente differente? (provate)

	100 B	10 kB	1 MB	100 MB
DES-CBC				
DES-EDE3				
RC4				
AES-128-CBC				
AES-192-CBC				
AES-256-CBC				
ChaCha20				

Tabella 1: *prestazioni di alcuni algoritmi di cifratura simmetrica.*

OpenSSL include un comando per verificare le prestazioni degli algoritmi di cifratura supportati. Provate ad eseguire il seguente comando per misurare le prestazioni di AES-128-CBC:

```
openssl speed aes-128-cbc
```

Quali differenze ci sono tra l'uso del comando `time` e quello del comando `speed` per valutare le prestazioni di un algoritmo crittografico?

→

2 Algoritmi di cifratura asimmetrica

2.1 Generazione di chiavi RSA

Usando il comando `genrsa`, scrivete la riga di comando per generare una coppia di chiavi RSA a 2048 bit, salvandola nel file `rsa.key.`*nome*, dove *nome* è il nome della persona che sta creando la chiave (es. Alice o Bob):

→

Ora analizzate il contenuto di `rsa.key.`*nome* lanciando:

```
openssl rsa -in rsa.key.nome -text
```

Riuscite a riconoscere la natura e lo scopo dei parametri `modulus`, `publicExponent`, `privateExponent`, e `prime*`?

→

Quali di questi costituiscono la chiave privata RSA e quali la chiave pubblica?

→

Confrontate i parametri con quelli generati da un vostro collega: ve ne sono in comune? Perché?

→

Supponete di voler distribuire la vostra nuova chiave pubblica ai vostri colleghi: scrivete i comandi OpenSSL per estrarre la chiave pubblica nel file `rsa.pubkey.`*nome* e visualizzarne il contenuto:

→

2.2 Operazioni di cifratura e decifratura asimmetrica con l'algoritmo RSA

Lo scopo di questo esercizio è usare la chiave RSA generata in precedenza per cifrare e decifrare. Create un breve messaggio di testo, ad esempio "*Un grandissimo segreto*", e salvatelo in un file chiamato `plain`.

Come potete usare la vostra nuova chiave RSA per proteggere la confidenzialità di `plain`? Quale chiave dovete usare? Scrivete il comando OpenSSL per cifrare `plain` e salvare il risultato nel file `encRSA` (suggerimento: usate il comando `pkeyutl`).

→

Cosa fa il seguente comando? Quale chiave è stata usata?

```
openssl pkeyutl -encrypt -in plain -inkey rsa.key -out plain.enc.RSA.for.name
```

→

Scrivete di seguito il comando per decifrare il messaggio precedentemente cifrato:

→

Provate infine a scaricare da Internet il seguente file[1]

```
wget https://cacr.uwaterloo.ca/hac/about/chap8.pdf
```

e provate a cifrarlo con la chiave RSA generata. Quali difficoltà avete incontrato? Perché?

→

NOTA

RSA permette in teoria di cifrare tutti i "messaggi" che, interpretati come numero binario, siano minori del modulo. Tuttavia, lo standard PKCS#1 impone delle limitazioni aggiuntive dovute all'imbustamento, in particolare al padding. Se avete una chiave RSA lunga N byte, perché le operazioni di cifratura con OpenSSL possano essere svolte correttamente, i dati in chiaro non devono essere più lunghi di $N - 11$ byte.
A partire da PKCS#1 v2 (vedere RFC 2437), lo standard introduce un nuovo metodo di padding noto come OAEP (Optimal Asymmetric Encryption Padding). In questa tecnica, il messaggio iniziale viene *espanso* fino ad una certa lunghezza di bit facendo del padding con degli zeri, a cui viene concatenato un intero casuale. Successivamente vengono effettuate delle operazioni di xor con degli hash per produrre il vero messaggio finale che sarà poi cifrato con RSA. Questa tecnica rendere più sicura la cifratura (evita dei potenziali attacchi di tipo chosen-ciphertext), ma è ancora poco diffusa. Per attivare questa tecnica di padding si può usare il flag `-oaep` nei comandi delle OpenSSL.

Se avete scaricato il file `chap8.pdf`, provate a darci un'occhiata aprendolo con:

```
atril chap8.pdf &
```

[1] Se doveste aver problemi a scaricare da Internet, ad esempio per problemi tecnici in laboratorio, potete usare invece il file `/etc/apache2/apache2.conf`.

2.3 Operazioni di firma e verifica con l'algoritmo RSA

Il comando `pkeyutl` permette non solo di cifrare/decifrare blocchi di dati, ma anche di firmarli/verificarli. Qual è la differenza a livello di operazioni RSA?

→

Scrivete i comandi OpenSSL per firmare `plain` salvando la firma nel file `sig.`*nome*, e successivamente per verificare la firma stessa. Quali chiavi dovete usare per ognuna delle operazioni?

→

2.4 Generazione di chiavi EC

Usando il comando OpenSSL `ecparam`, scrivete la riga di comando per generare una curva ellitica "SECG curve over a 192 bit prime field", salvandola nel file `ec.key.`*name*, dove *nome* è il nome della persona che sta creando la chiave (es. Alice o Bob):

→

Supponete di voler distribuire la vostra nuova chiave pubblica EC ai vostri colleghi: scrivete i comandi OpenSSL per estrarre la chiave pubblica nel file `ec.pubkey.`*nome* e visualizzarne il contenuto:

→

2.5 Operazioni di firma e verifica EC

Il comando `pkeyutl` permette non solo di fare operazioni crittografiche con RSA ma anche di firmarli/verificarli con l'algoritmo ECDSA.

Scrivete i comandi OpenSSL `pkeyutl` per firmare il file `plain` con ECDSA, salvando la firma nel file `ecsig`, e successivamente per verificare la firma stessa. Quali chiavi dovete usare per ognuna delle operazioni?

→

2.6 Prestazioni

Usate il comando `openssl speed` per effettuare i seguenti confronti di prestazione tra:

- RSA a 512, 1024, 2048 e 4096 bit.

  Come diminuiscono le prestazioni?

  Perché le due operazioni RSA si comportano in modo radicalmente diverso?

→

- algoritmi di firma digitale a bassa sicurezza: RSA a 1024 bit, DSA a 1024 bit, ECDSA a 160 bit (`ecdsap160`).

  Che differenze notate?

  →

- algoritmi di firma digitale ad alta sicurezza: RSA a 2048 bit, DSA a 2048 bit, ECDSA a 256 bit (`ecdsap256`).

  > NOTA
  >
  > In realtà la crittografia su curve ellittiche a 256 bit garantisce una sicurezza pari a RSA/DSA con chiavi a 3072 bit, quindi il confronto è leggermente a sfavore degli algoritmi che usano curve ellittiche.

  Che differenze notate con i risultati del passo precedente?

  →

- crittografia asimmetrica, simmetrica e digest: RSA 2048 bit, DSA 2048 bit, AES a 128 bit, SHA-256.

  > NOTA
  >
  > Come spiegato nella nota precedente, AES a 128 bit garantisce una sicurezza pari a RSA/DSA a 3072 bit.

  Come confrontate i risultati ottenuti?

  Quale rapporto di massima esiste tra i costi della crittografia simmetrica e asimmetrica?

  →

3 Algoritmi di digest

3.1 Calcolare e verificare il digest di un messaggio

Create un messaggio di testo tipo "*Questo e' un messaggio di prova per il calcolo del digest!*" in un file chiamato `msg` e calcolate il suo digest usando gli algoritmi MD5, SHA1, SHA-256, SHA3-256. Salvate il risultato nei file `MD5dgst`, `SHA1dgst`, `SHA256dgst`, e `SHA3-256dgst`. Scoprite i comandi OpenSSL da lanciare e scriveteli qui:

→

Ora provate a modificare il messaggio (ad esempio rimuovendo il "!" finale) ed a ricalcolarne il digest. Cosa si può notare confrontando i due digest?

→

Supponete ora di voler calcolare un digest su `msg` usando l'algoritmo AES: quale schema potreste usare?

→

3.2 Collisioni

Proviamo ora a valutare la robustezza di un algoritmo di digest e quanto conta la sua lunghezza in bit presentando un esempio di algoritmo di hash "insicuro".

Il file `IHA.sh`, presente nell'archivio contenente il materiale di supporto, è un esempio di algoritmo di digest forte limitato solo dal fatto che la lunghezza del digest è molto piccola. Più precisamente, IHA produce un digest lungo 4 bit.

Il file `IHA.sh` accetta come primo parametro da riga di comando il nome del file di cui calcolare il digest:

```
bash IHA.sh file_input
```

Calcolate quindi il digest di `msg` con IHA.

Identificate una procedura per creare un altro messaggio che elaborato con IHA dia come risultato un digest identico. Quante volte sarà necessario eseguire IHA in media?

→

Supponete di voler cercare una collisione arbitraria, cioè due messaggi che abbiano lo stesso digest (e non un messaggio che abbia il digest uguale a quello di un altro messaggio fisso. Quanto dovrebbe essere lungo il digest per richiedere mediamente lo stesso numero di tentativi impiegato nell'operazione precedente? Riuscite ad immaginare una situazione in cui poter ricavare una collisione arbitraria risulti in una seria minaccia?

→

3.3 Prestazioni

Ora provate a valutare il costo associato agli algoritmi di digest implementati da OpenSSL, seguendo la metodologia usata nell'Esercizio 1.3 per gli algoritmi di cifratura. Confrontate i risultati ottenuti con quelli precedentemente calcolati per gli algoritmi di cifratura simmetrici e compilate la Tabella 2.

→

	100 B	10 kB	1 MB	100 MB
MD5				
SHA1				
SHA256				
SHA512				
SHA3-256				
SHA3-512				

Tabella 2: *tempo di elaborazione di alcuni algoritmi di digest.*

4 Approfondimenti ed ulteriori esercizi (opzionali)

Esercizi aggiuntivi sulla crittografia simmetrica

4.1 Modifica dei messaggi cifrati con crittografia simmetrica

Proviamo a modificare un messaggio cifrato con la crittografia simmetrica. Dobbiamo simulare una comunicazione tra due persone, Alice e Bob, tramite un canale in cui Mallory, attaccante "attiva", intercetta e modifica i messaggi.
Immaginate, ad esempio, che Alice debba inviare a Bob un numero, scritto in un file di testo, che rappresenta l'importo del bonifico che Alice chiede a Bob di inviare a Mallory, che è ovviamente interessata ad aumentarne il valore. Supponiamo inoltre che Alice e Bob abbiamo un segreto condiviso da usare come chiave simmetrica (e se non ce l'hanno se la scambiano con metodi sicuri che non siamo interessati ad implementare in questo contesto).
Questo è lo scenario che vogliamo simulare:

1. Alice crea il file di testo col comando:

   ```
   echo 456789 > numeri.txt
   ```

2. Alice cifra `numeri.txt` con l'algoritmo RC4 usando il segreto condiviso con Bob ottenendo il file cifrato `numeri.enc`. Quale è il comando che deve lanciare Alice?

   ```
   →
   ```

3. Mallory intercetta `numeri.enc` mentre transita da Alice a Bob, ma prima di recapitarlo a Bob, prova a modificarlo. Dato che Mallory non conosce la chiave comune prova ad eseguire questo attacco:

   (a) apre `numeri.enc` usando l'editor esadecimale `hexedit`:

   ```
   hexedit numeri.enc
   ```

   (b) cambia alcuni bit in `numeri.enc`, salva ed esce da `hexedit` (`Ctrl+X` per salvare ed uscire)

4. Bob riceve `numeri.enc`, lo decifra e cerca di capire se il messaggio recapitatole da Mallory è stato modificato o meno.

Il file cifrato modificato da Mallory contiene ancora numeri decimali? Cosa può fare Mallory per ingannare Alice and Bob ottenendo ancora un file con un numero decimale (o almeno aumentare la probabilità

di avere un numero decimale)?

→

Cosa cambia usando AES-128-ECB o AES-128-CBC al posto di RC4?

→

4.2 Uso degli algoritmi di cifratura a blocchi

Prendete in considerazione i tre file `ecb*` nel materiale di supporto. I due file `ecb_plain_1` e `ecb_plain_2` contengono due possibili messaggi in chiaro, mentre `ecb_enc` è la cifratura di uno dei primi due col seguente comando:

```
openssl enc -e -in ecb_plain_X -out ecb_enc -K chiave -iv 0 -des-ecb
```

Riuscite a scoprire quale dei due messaggi è il file originale che è stato cifrato?

→

Esercizi aggiuntivi sugli algoritmi di digest

4.3 Integrità di file e cartelle

I programmi `sha1sum` e `hashdeep` consentono di calcolare comodamente l'hash di uno o più file (il secondo elabora ricorsivamente i file contenuti in una cartella con uno o più algoritmi a scelta).
Create una cartella, chiamatela `tree`, ed al suo interno create un albero di sotto-cartelle e file; per semplicità potete utilizzare lo script di test `gen_tree.sh` nel materiale di supporto.
Scrivete qui di seguito il comando per calcolare il digest di tutti i file contenuti nella cartella `tree`

→

cambiate il contenuto di un file (es. aggiungendo uno spazio al fondo) e verificate cosa succede con

```
hashdeep -c sha1 -r -x -k hash_list tree
```

Cosa può fare un attaccante per cambiare il contenuto di alcuni file e non farsi scoprire?

→

Quali accorgimenti si possono adottare per non essere imbrogliati? Pensatene almeno due.

→